



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۱۳ آبان

فصل سوم: لایه *Transport* اسلاید ۵۵ تا ۸۸ [۱]

TCP

- این پروتکل معمولاً *point to point* است.
- پیام به صورت *byte stream* است، یعنی اولاً مرزبندی بین پیام‌ها وجود ندارد و ثانیاً پیام به صورت بایت به بایت منتقل می‌شود.
- انتقال *byte stream* به صورت *reliable* و *in - order* است.
- ارسال سگمنت‌ها به صورت *pipeline* است و به اندازه پنجره می‌توان داده در پرواز یا در لوله داشت. اندازه این پنجره به صورت دینامیک تحت تأثیر *flow control* و *congestion control* است.
- اتصال *fullduplex* است یعنی دوطرف روی یک اتصال می‌توانند همزمان به هم داده ارسال کنند.
- هر سگمنت بر حسب بایت، دارای یک حداکثر طولی است که به آن $MSS = maximum\ segment\ size$ می‌گویند.
- *connection - oriented* است و *handshake* اولیه صورت می‌گیرد. برای رد و بدل شدن پیام‌های کنترلی و یک سری متغیر و ساختمان داده‌ها و ...
- *flow control* داریم برای پر نکردن بافر مقصد و *congestion control* برای کنترل شلوغی خود شبکه و جلوگیری از ارسال داده زیاد در این هنگام است.

بخش‌های مختلف سگمنت *TCP*

- یک قسمت برای هدر و یک قسمت *payload* یا همان داده اصلی کاربر
- پورت مبدا و پورت مقصد
- *sequence number* اولین بایت آبی (بایت‌های ارسال نشده آماده برای ارسال) در اسلاید ۵۸
- *acknowledgement number* شماره ترتیب بایت بعدی؟
- *header length* یا *data offset* مقدار پیش فرض آن ۵ است یعنی ۵ ضرب در ۳۲ بیت. ۴ بیت است. با آن اندازه *option* را می‌توان مشخص کرد.

- فیلد *option* که میتواند تا ۴۰ بایت داشته باشد.
 - یه بخش بدون استفاده داره
 - *urgent data pointer* پوینتر به انتهای داده فوری
 - *recieve window* ۱۶ بیت است و مقصد مقدار بافر خود را از این طریق به مبدا میگوید.
 - *flag U* یعنی *urgent* یعنی سریع تر سگمنت را مطالعه کند.
 - *flag A* یعنی سگمنت *ACK* است.
 - *flag P* به معنی *push* است و به مبدا میگوید که سگمنت جاری را سریع تر به شبکه هل دهد.
 - *flag R* یا *reset*
 - *flag S* یا همان *synchronized*. برای زمانی که مبدا میخواهد به مقصد وصل شود.
 - *flag F* برای درخواست خاتمه اتصال
- دو تا فلگ *S* و *A* برای برقراری قبول اتصال کاربر در مقصد و فلگ *R* در صورت عدم قبول اتصال توسط سرور
 فلگ های *R*، *S* و *F* برای مدیریت اتصال
 فیلد *checksum* برای تشخیص خطا

اسلاید ۵۹

فیلد *ACK* در *host A*، با توجه به *sequence number* در *host B* در دور قبلی مشخص میشود. (یکی بیشتر از اونه)

فیلد *ACK* هر هاست با توجه به *sequence number* هاست مقابل انتخاب میشود. و سگمنت بعدی را هم درخواست میکند.

مثلا *ACK ۸۰* میفرسته یعنی من تا سر ۸۰ رو گرفتم و از ۸۰ به بعد رو بفرس برام. قبلش کاربر تا ۷۹ فرستاده بوده که این توی *sequence number* مشخص شده.

مقدار *timeout* باید از *RTT* یکم بزرگتر باشد (نه خیلی بزرگ) که بهش میگویند *safety margin*

اگر کوچک تر باشد، *premature* باعث ارسال چند باره دیتا میشود.

اگر خیلی بزرگ تر باشد، باعث عکس العمل کند در مقابل اتلاف بسته

tcp مقدار *rtt* رو تخمین میزنه. یه متغیر *sampleRTT* برای زمان ارسال سگمنت جاری تا دریافت *ACK* آن. میانگین چندتا *sampleRTT* اخیر رو میگیریم چون خیلی هم تغییر پذیر است این متغیر. این متغیر برای تغییرات لحظه ای است

اثر تاریخچه *RTT* ها در متغیر *EstimatedRTT* ذخیره می‌شود.

$$EstimatedRTT = (1 - \alpha) * EstimatedRTT + \alpha * SampleRTT$$

مقدار α معمولاً $2^{-3} = 0.125$ است. برای همین نمایی کم میشود.

گفتیم باید $timeout$ از $EstimatedRTT$ بیشتر دارد. حالا چقدر؟ بستگی به میزان انحراف و تغییرات $EstimatedRTT$ از $sampleRTT$ دارد.

$$DevRTT = (1 - \beta) * DevRTT + \beta * |SampleRTT - EstimatedRTT|$$

معمولاً $\beta = 0.25$ و در نتیجه برای محاسبه $timeout$ داریم:

$$TimeoutInterval = EstimatedRTT + 4 * DevRTT(safety\ margin)$$

پیاده سازی rdt در TCP

- با استفاده از مکانیزم های (۱) تایمر بازارسال منفرد (؟) و (۲) استفاده از *pipeline* و همچنین (۳) *cumulative ACK* با *ACK* های تکراری و یا *timeout* داده را *retransmit* میکنیم.
- اگر سگمنت *ACK* شود، همه سگمنت ها تا اون *ACK* میشن انگار.
- در *lost ACK* و بعد از *timeout* باز ارسال میشود.
- در حالت *premature* قدیمی ترین سگمنت باز ارسال می شود، و ولی باز *ACK* آخرین سگمنت یا همون *ACK* تجمعی ارسال می شود. (یعنی اگر *ACK* بسته ۱۰۰ تلف بشه ولی برای ۱۲۰ برسه، یعنی ۱۰۰ رو هم گرفته قبلاً)
- اگر سگمنت خارج اردر باشه، گیرنده *duplicate ACK* میفرسته.
- اگر بسته که میفرستیم همه قبلی هاش *ACK* شده باشن، برای اون بسته *delayed ACK* داریم به اندازه $500ms$.
- اگر یه بسته بفرسته و از قبل هم یه بسته بوده که هنوز *ACK* ش نیومده و *pending* است، گیرنده هر دو را با *ACK* تجمعی، اک میکند.
- اگر سگمنتی بفرستند که گپ را پر کند، *ACK* تجمعی تا آنجا میفرستد (چجوری؟) و اون سگمنت ها رو به کاربرد میده.

مکانیزم fast retransmit در TCP

با دریافت *ACK* تکراری مثلاً ۳ تا تکراری، فرستنده زودتر از *timeout* اون سگمنت را باز ارسال میکند.

TCP flow control

بافر های سوکت های گیرنده ممکن است پر شود.

اندازه کل بافر: *rcvbuffer* و مقدار خالی بافر: *rwnd* که میتونیم این مقدار رو در *ACK* منتقل کنیم به فرستنده. مسئله *silly window* چیست؟

بعد از *handshake* اولیه، وارد حالت *stablishment* میشوند.

عدم کفایت *handshake*: به دلیل تاخیر متغیر در شبکه و خارج از ترتیب رسیدن پیام‌ها و پیام های بازارسال در اثر اتلاف.

ممکنه پیام باز ارسال اتصال جدید، با تاخیر زیاد بعد از بسته شدن اتصال اول برسه و یه اتصال نصفه از سمت سرور باز میشه. (اگر داده را باز ارسال کنه بعدش و با تاخیر توی اتصال دوم برسه، دیگه بدتر) اسلاید ۷۷ خیلی مهم بنظر میاد. اول هر دوی مشتری و سرور در حالت *LISTEN* هستن. بعد مشتری یک سگمنت *SYN* با یه *sequence number* رندوم مثل x میفرستد با $SYNbit = 1$ که همون فلگ S است. مشتری وارد استیت *SYNSENT* میشود.

در جوابش، سرور که در حالت *LISTEN* بود، با گرفتن پیام، یک سگمنت *SYNBACK* با $SYNbit = 1$ و $ACKbit = 1$ میفرستد که یک *sequence number* رندوم مثل y میفرستد و $ACKnum = x + 1$ دارد یعنی تا بسته x را گرفته و منتظر سگمنت $x + 1$ از مشتری است. وقتی که اینو فرستاد وارد استیت *SYNRCVD* میشه. بعد مشتری پیام رو دریافت میکنه و وارد استیت *ESTAB* میشه. و در جواب $ACKbit = 1$ میفرستد. چرا واقعا؟) و برای $ACKnum = y + 1$ میفرستد. که یعنی تا سگمنت y رو مشتری داره. سرور که این پیام رو میگیره، وارد *ESTAB* میشه.

برای قطع اتصال، هر کدام از سرور یا مشتری باید فلگ F رو به ۱ ست کنند. در جواب باید یک ACK با $FIN = 1$ بفرسته اون طرف.

congestion control

اگر اندازه بافر مقصد را بی نهایت فرض کنیم، ازدحام حداقل باعث تاخیر در ارسال بسته ها می‌شود. اگر بافر ها متناهی باشند، با *packet loss* و در نتیجه *retransmission* نیز مواجهیم.

درواقع اگر نرخ انتقال داده را λ_{in} بگیریم، نرخ اصلی انتقال داده (با در نظر گرفتن *retransmission*) برابر λ'_{in} است که $\lambda'_{in} \geq \lambda_{in}$ است. نرخ بسته ها در *tcp* به خاطر همین باز ارسال ها، بیشتر از λ_{in} عادی است.

اگر ما *perfect knowledge* داشته باشیم از بافر روتر ها، خب سعی میکنیم که $\lambda_{in} = \lambda_{out}$ باشد و تنها زمانی ارسال میکند که بافر روتر ها خالی باشد. پس بسته ای تلف نمی شود و همه بسته ها ارسال می شود.

اگر *some perfect knowledge* داشته باشیم، یعنی مبدا وضعیت پر یا خالی بودن بافر روتر ها را نمیداند، ولی اگر یک بسته تلف شود، مبدا میداند که تلف شده و باز ارسال میکند. پس در نهایت بسته ها تنها یک بار به مقصد میرسند (با اینکه مبدا دو بسته ارسال کرده اما مقصد قطعا یکی گرفته، یعنی باز ارسال فقط برای بسته های تلف شده در شبکه) و دقیقا به خاطر همین موضوع، اگر $\lambda'_{in} = R/2$ باشد، $\lambda_{out} < R/2$ است چون مثلا بعضی بسته ها که دوبار ارسال شده یه بار به مقصد رسیده. در یک نقطه $\lambda'_{in} > R/2$ به $\lambda_{out} = R/2$ خواهیم رسید بالاخره.

در حالت سوم، شرایط واقعی است، و مبدا از بافر روتر هیچ اطلاعاتی ندارد، به دلیل تلف شدن بسته و یا *timeout* شدن و ارسال دوباره بسته هایی که حتی تلف نشده بودند. توی این حالت بخاطر همین بسته های نامفید، هیچوقت $\lambda_{out} = R/2$ نخواهد شد. و کمتر از آن است همیشه.

ناحیه *congestion collapse*

اسلاید ۸۷ خیلی خیلی مهمه، حتما بعدا نگاه کن یا از استاد بپرس. اینطور میگفت که اگر جریان قرمز را زیاد کنیم، چون میتوانیم هر چقدر بخوایم، جریان قرمز را زیاد کنیم اما جریان آبی حداکثر میتواند $R/2$ باشد. اگر جریان قرمز را زیاد کنیم، جریان آبی به صفر میل می کند.

درواقع مثلا جریان آبی تا یه جایی زیاد میشود، از یه جایی به بعد که λ'_{in} زیاد شود، جریان آبی خیلی کم، زیاد میشود و اگر باز بیشتر شود، جریان آبی به صفر میل می کند.

مراجع

[1] “ch3-v8-slide-55-to-88,” <https://drive.google.com/drive/folders/>, 2022.